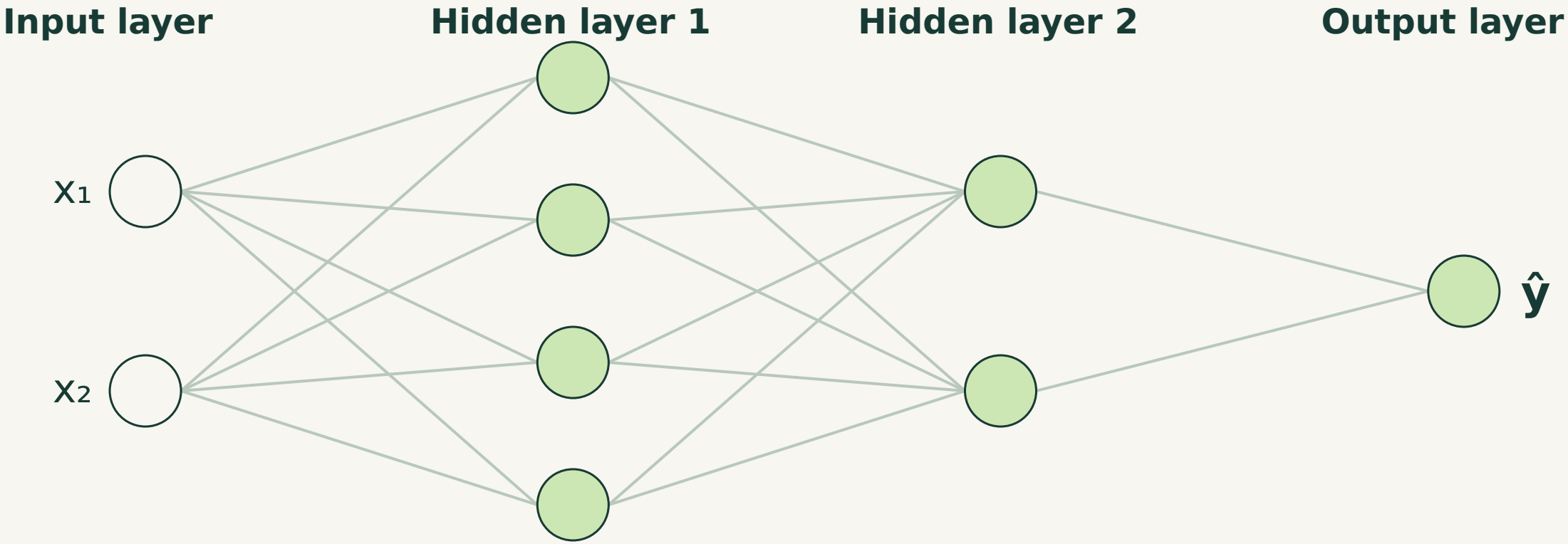
Neural network components

How a network makes a prediction and learns from its errors

Pavan Naga Phanendra Kancheti

Architecture, core components, and TensorFlow Playground observations

Layers organize the flow of information



2 features | 4 neurons | 2 neurons | 1 class score

Forward pass → Each hidden layer transforms the preceding layer’s outputs.

A neuron combines inputs and applies an activation

$$z = w_1x_1 + w_2x_2 + b$$

$$a = f(z)$$

Worked example

$x_1 = 2$, $x_2 = 1$
$w_1 = 0.5$, $w_2 = -1$, $b = 0.2$

Result with ReLU

$z = 1 - 1 + 0.2 = 0.2$
$a = \max(0, 0.2) = 0.2$

Weights control influence; bias shifts the threshold

Positive weight

An input contributes in the same direction to the weighted sum.

Negative weight

An input contributes in the opposite direction to the weighted sum.

Bias

A learned offset changes the neuron’s response even when inputs are zero.

Training updates weights and biases. Their magnitudes depend on feature scale.

Activation functions enable nonlinear patterns

Function	Transformation / range	Role or limitation
ReLU	max(0, z) · [0, ∞)	Common hidden activation
Sigmoid	1 / (1 + exp(−z)) · (0, 1)	Binary probability output*
Tanh	tanh(z) · (−1, 1)	Used in these Playground runs
Linear	z · all real values	No added nonlinearity

Stacking linear layers alone still yields a linear mapping.

*With appropriate training. Playground classification uses a tanh score, not a probability.

Loss measures how far predictions are from targets

Playground loss = average of ½(ŷ − y)²

One example

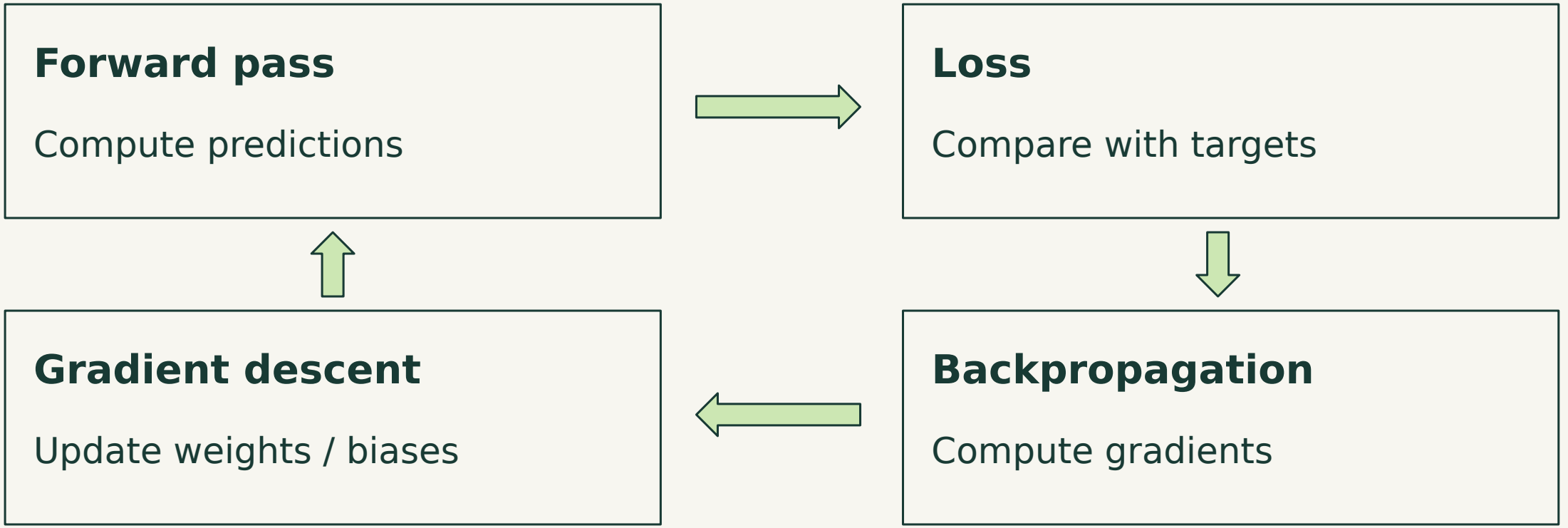
Target y = 1; prediction ŷ = 0.8
Loss = ½(0.8 − 1)² = 0.02

Reading the display

Training loss uses examples for learning.
Test loss uses held-out examples.

Lower loss is better for this objective. Loss 0.02 does not mean 98% accuracy.

Optimization updates parameters to reduce loss



w ← w − learning rate × average gradient in the batch

Experiment settings and comparison plan

Shared settings

Classification · learning rate 0.03
Batch size 10 · training split 50%
No regularization · tanh activation

Changes explored

Circle versus XOR datasets
Noise 0 versus 50
Two hidden layers versus none
Raw inputs versus product feature

Epoch = one pass through the training examples.

Circle runs use 100 epochs. XOR observations record their actual checkpoints.
Single runs demonstrate behavior; they do not establish a universal ranking.

Measured results in TensorFlow Playground

Run / data	Hidden layers	Inputs	Epoch	Train loss	Test loss
A · circle, noise 0	4 → 2	x_1, x_2	100	0.016	0.017
B · circle, noise 50	4 → 2	x_1, x_2	100	0.320	0.327
C · XOR, noise 0	None	x_1, x_2	133	0.510	0.500
D · XOR, noise 0	None	+ x_1x_2	145	0.004	0.007

A/B seeds: 0.36469 / 0.71571. C/D seed: 0.98530. Values rounded by the UI.

No wall-clock timing or accuracy measurement was collected.

Features and noise change the learning problem

Circle: higher noise, higher loss

Test loss rose from 0.017 to 0.327. Noise makes the classes harder to separate; different seeds also limit causal attribution.

XOR: a useful feature matters

A raw-input model without hidden layers has a straight decision boundary. The product x_1x_2 exposes the quadrant relationship.

Complexity: A/B have 25 trainable parameters; C has 3; D has 4.

More parameters increase work per update, but do not guarantee lower test loss. Repeated trials and a fixed timing protocol would be needed to compare speed.

Learning reflection

Reviewing the Playground demonstrations clarified how a network’s parts work together. Layers define the structure, but useful input features and nonlinear activations determine which relationships it can represent.

The XOR example made feature design concrete: adding a product feature changed what a very small model could learn. The noisy circle example also showed why evaluating held-out data matters.

For my interests in AI and networking, the next step is to test these ideas on traffic measurements, compare repeated runs, and report limitations alongside performance.

References

Google. (2025). Neural networks: Nodes and hidden layers.

developers.google.com/machine-learning/crash-course/neural-networks/nodes-hidden-layers

Google. (2025). Neural networks: Activation functions.

developers.google.com/machine-learning/crash-course/neural-networks/activation-functions

Google. (2025). Neural networks: Training using backpropagation.

developers.google.com/machine-learning/crash-course/neural-networks/backpropagation

TensorFlow. (n.d.). A neural network playground.

https://playground.tensorflow.org/

TensorFlow. (n.d.). Playground [Source code]. GitHub.

https://github.com/tensorflow/playground

Summary: structure, learning, and evidence

Structure

Layers connect neurons. Weights and biases control each neuron’s input.

Learning

Activation functions add nonlinearity. Loss and optimization guide parameter updates.

Visualization makes the relationships visible: data flow explains predictions;
the training loop explains how those predictions change.

The strongest model choice depends on the data, features, and measured test behavior.